
PROJECT 3: SPARSE NEURAL NETWORK TRAINING AND OPTIMIZATION*

Ammy Lin

Tonantzin Real Rojas

Anvita Suresh

ABSTRACT

This project studies optimization dynamics in sparse neural networks trained from scratch using ResNet-20 on CIFAR-10. We evaluate Static Sparse Training, Sparse Evolutionary Training (SET), and Rigging the Lottery (RigL) at 90%, 95%, and 98% sparsity, comparing SGD with momentum and Adam. Results show that static masks provide a stable baseline, while dynamic sparsity improves performance only when guided by gradients. RigL consistently outperforms SET due to gradient-informed rewiring. Adam is more robust under dynamic sparsity, while SGD performs well in static settings but degrades at high sparsity. Overall, performance depends strongly on sparsity level and optimizer choice.

1 INTRODUCTION

Modern deep learning models are heavily over-parameterized, often containing many redundant weights that increase computational and memory cost. Prior work has shown that high accuracy can still be achieved with significantly fewer active parameters, motivating sparse training approaches that operate directly under constrained connectivity. In sparse training, a binary mask M is applied to model parameters θ , producing effective weights:

$$W = M \odot \theta,$$

where inactive connections remain fixed at zero.

This work studies sparse training from scratch using a ResNet-20 model on CIFAR-10 under Static Sparse Training, Sparse Evolutionary Training (SET) (Mocanu et al., 2018), and RigL (Evci et al., 2020). We evaluate all methods at 90%, 95%, and 98% sparsity to study optimization under extreme parameter reduction. We additionally compare SGD with momentum and Adam to assess their behavior under static and dynamically changing sparse connectivity.

2 MODELS AND DATASET

All experiments use CIFAR-10 (50k train / 10k test), with images normalized using standard channel statistics. No data augmentation is used to isolate the effects of sparsity and optimization.

2.1 MODEL ARCHITECTURE

We use a ResNet-20-style architecture for CIFAR-10 with an initial 3×3 convolution followed by three residual stages (16→32→64 channels). Each stage contains three BasicBlocks with skip connections. The model has approximately 0.27M parameters.

2.2 SPARSITY SETTINGS

We evaluate models at 90%, 95%, and 98% sparsity using binary masks applied to all convolutional and linear layers. Static masks remain fixed, while SET and RigL dynamically update connectivity. Both SGD with momentum and Adam are evaluated under identical settings.

Optimizers: SGD uses learning rate 0.1 with momentum 0.9; Adam uses learning rate 0.001. All models are trained for 15 epochs with batch size 128 across six configurations (3 sparsity levels $\times$ 2 optimizers).

*https://github.com/ammylin/ECE685D_Final_Project.git

2.3 EVALUATION METRICS

Performance is measured using test accuracy and training/test loss curves across epochs.

Sparsity is defined for a weight tensor W as:

$$\text{sparsity}(W) = 1 - \frac{\|W\|_0}{|W|}.$$

3 METHODOLOGY

3.1 STATIC SPARSE TRAINING

We first evaluate static sparse training, where a fixed binary mask is applied from initialization and remains unchanged. This isolates sparsity effects under standard optimization.

3.2 SPARSE EVOLUTIONARY TRAINING (SET)

SET enables dynamic sparse connectivity by periodically rewiring the network while maintaining constant sparsity (Mocanu et al., 2018). At each epoch, a fraction of low-magnitude weights is pruned and replaced with randomly sampled connections. Newly added weights are initialized using a scaled normal distribution to preserve stability. This prune-and-regrow process maintains a fixed sparsity budget while continuously altering the network topology.

3.3 RIGGING THE LOTTERY (RigL)

RigL improves SET by replacing random regrowth with gradient-guided selection (Evci et al., 2020). Instead of sampling new connections randomly, RigL activates weights with the largest gradient magnitude, targeting parameters that most reduce loss. This ensures that topology updates are both structured and loss-aware while preserving constant sparsity.

4 EXPERIMENTAL RESULTS

4.1 RESULTS BY METHOD

4.1.1 STATIC SPARSE TRAINING

Figure 1 illustrates the impact of increasing sparsity on final test accuracy under Static Sparse Training. Across both optimizers, performance degrades as the proportion of pruned weights increases, with a consistent but non-linear decline.

- **90%:** SGD 70.1%, Adam 73.3%.
- **95%:** SGD 61.5%, Adam 57.9%.
- **98%:** SGD 50.5%, Adam 47.3%.

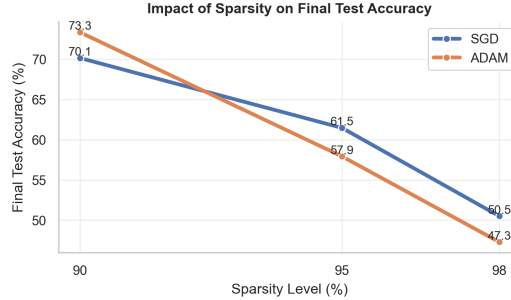


Figure 1: Final Test Accuracy vs. Sparsity for Static Sparse Training

The degradation in accuracy is non-linear, with a sharper drop from 95% to 98% sparsity, indicating that extreme pruning begins to remove essential representational capacity.

4.1.2 SPARSE EVOLUTIONARY TRAINING (SET)

The performance of SET is highly sensitive to both sparsity level and optimizer choice, exhibiting non-linear and in some cases unstable behavior across configurations (Table 1).

Table 1: Final Test Accuracy Summary for SET

Optimizer	90%	95%	98%
Adam	33.27%	35.91%	42.40%
SGD	25.52%	10.07%	46.57%

A key failure case occurs at 95% sparsity for SGD, where test accuracy collapses to 10.07%. Training dynamics in this setting are highly unstable, with large fluctuations in test loss and frequent divergence. This suggests that aggressive pruning and frequent rewiring prevent stable connectivity.

Interestingly, performance improves at 98% sparsity for both optimizers, with SGD reaching 46.57%. However, this effect is likely stochastic and dependent on favorable initialization and rewiring trajectories, rather than a consistently stable training regime.

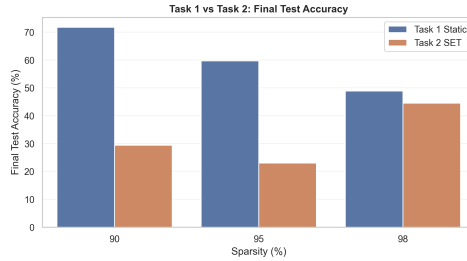


Figure 2: Final Test Accuracy: Static Sparse Training vs. SET

Across all sparsity levels, SET underperforms the static baseline. Random regrowth introduces poorly targeted connections, and frequent rewiring prevents stable convergence. The performance gap is largest at 90% and 95%, but narrows at 98%.

4.1.3 RIGGING THE LOTTERY (RigL)

Figure 3 presents the final test accuracy of RigL across sparsity levels and optimizers. Overall, RigL achieves strong performance at moderate sparsity but exhibits a pronounced optimizer-dependent divergence at extreme sparsity.

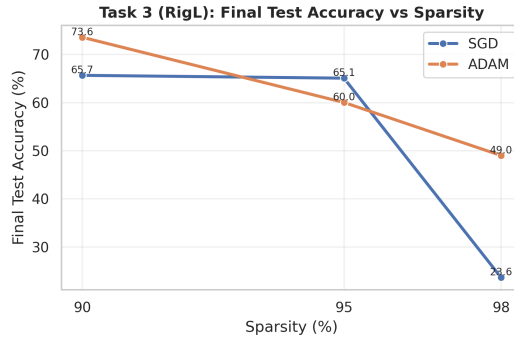


Figure 3: RigL Final Test Accuracy vs Sparsity

- **90% Sparsity:** Adam achieves 73.6% accuracy, outperforming SGD at 65.7%.

- **95% Sparsity:** The performance gap narrows, with SGD reaching 65.1% and Adam 60.0%. Notably, SGD maintains performance close to its 90% level, suggesting that gradient-guided rewiring helps preserve useful subnetworks under increased sparsity.
- **98% Sparsity:** A sharp divergence emerges (Figure 4). Adam remains stable at 49.0% test accuracy, while SGD collapses to 23.6%. Despite continued increases in training accuracy (over 60%), SGD exhibits severe generalization failure, indicating overfitting to transient sparse connectivity patterns.

At 98% sparsity, SGD collapses while Adam remains stable. This suggests momentum is destabilized by frequent topology changes, whereas Adam’s adaptive updates provide robustness under dynamic sparsity.

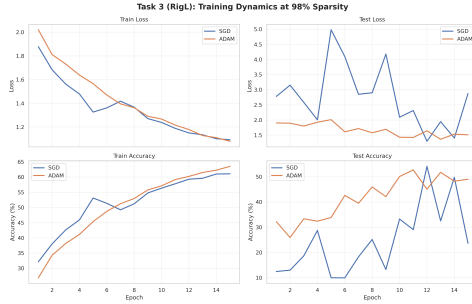


Figure 4: RigL Test Accuracy at 98% Sparsity

4.2 OPTIMIZER COMPARISON

Figure 5 shows training and test performance across sparsity levels under Static Sparse Training.

At 90% sparsity, both optimizers perform well despite heavy pruning. Adam achieves slightly lower training loss and marginally higher test accuracy, while SGD converges faster but is more variable. Overall, performance is comparable, with Adam offering more stable generalization.

At 95% sparsity, optimization becomes harder. SGD attains lower training loss and higher peak test accuracy but with greater fluctuation, while Adam remains more stable yet slightly underperforms in final accuracy.

At 98% sparsity, both optimizers struggle due to extreme compression. SGD shows lower training loss but higher variance, while Adam is smoother but achieves lower accuracy, indicating reduced model capacity.

Overall, SGD performs better in static training, whereas Adam provides more stable generalization, with this gap widening under dynamic sparse training due to SGD’s sensitivity to topology changes.

4.3 STATIC VS. DYNAMIC SPARSITY

Figure 6 summarizes the final test accuracy across all three methods at each sparsity level. The delta heatmaps in Figure 7 provide a direct quantitative comparison of RigL’s accuracy gain or loss relative to each baseline.

RigL vs. SET: Across all sparsity levels and optimizers, RigL significantly outperforms SET, with average gains of 23.7 pp (Adam) and 24.1 pp (SGD). This improvement stems from gradient-guided rewiring, which selects connections more likely to reduce loss, leading to better convergence and more effective sparse structures. The only exception is SGD at 98% sparsity, where RigL (23.7%) underperforms SET (46.6%), due to SGD instability under extreme dynamic sparsity rather than failure of the rewiring strategy.

RigL vs. Static Sparse Training: The comparison is optimizer-dependent. For Adam, RigL consistently matches or slightly exceeds the static baseline (+0.2 pp at 90%, +2.1 pp at 95%, +1.7 pp at 98%), yielding a small average gain (+1.4 pp). For SGD, RigL underperforms at 90% (4.5 pp)

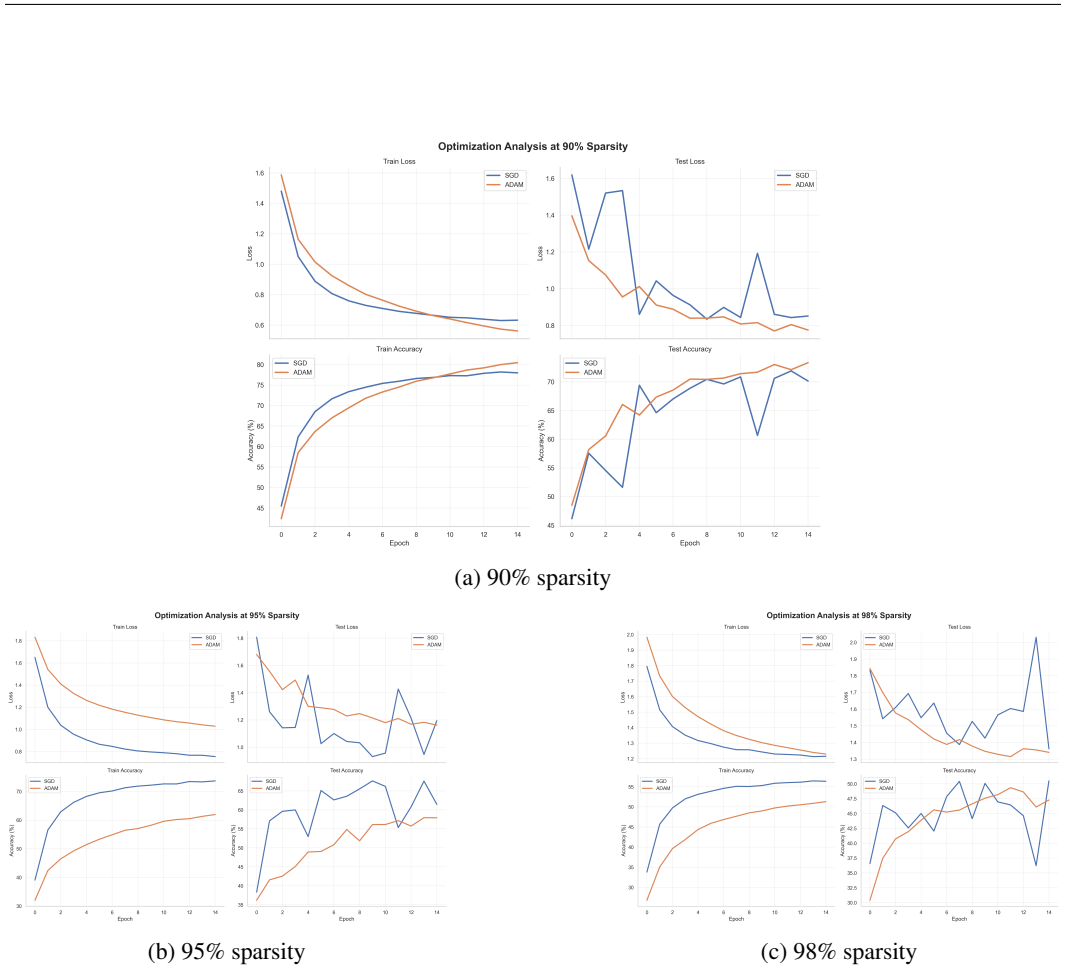


Figure 5: Training and test loss/accuracy curves for SGD and Adam under different sparsity levels during Static Sparse Training.

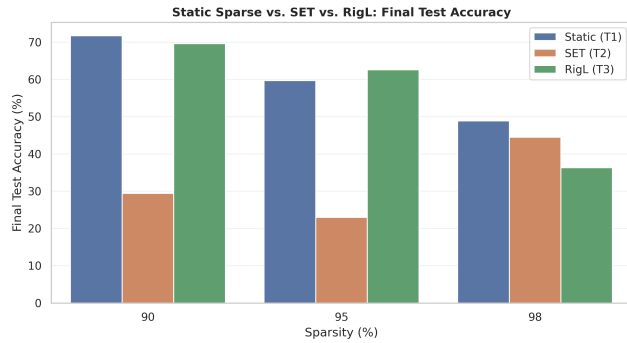


Figure 6: Final Test Accuracy across all 3 methods

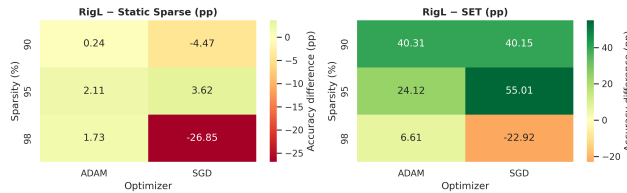


Figure 7: RigL's accuracy gain or loss relative to each baseline

and 98% (26.9 pp), with a partial recovery at 95% (+3.6 pp), resulting in an overall deficit (9.2 pp). This suggests Adam benefits from dynamic topology updates, while SGD is more stable with fixed sparsity.

Overall, RigL consistently improves over SET, confirming the benefit of gradient-guided sparsity. Its advantage over static training depends on the optimizer: Adam benefits from dynamic sparsity, whereas SGD is more competitive in static settings but degrades under frequent rewiring and extreme sparsity.

4.4 SPARSITY DISTRIBUTION AND KERNEL VISUALIZATION

To examine how sparsity is distributed across the network and how different training methods affect the structure of learned filters, we visualized the 16 stem convolutional filters (conv1) for Static Sparse Training, SET, and RigL, all trained with SGD at 90% sparsity. Each filter is displayed as a 3×3 patch averaged over the three RGB input channels, where red indicates positive weight values, blue indicates negative weight values, and white regions correspond to pruned connections. The percentage of active connections within each filter is reported in the subplot title.

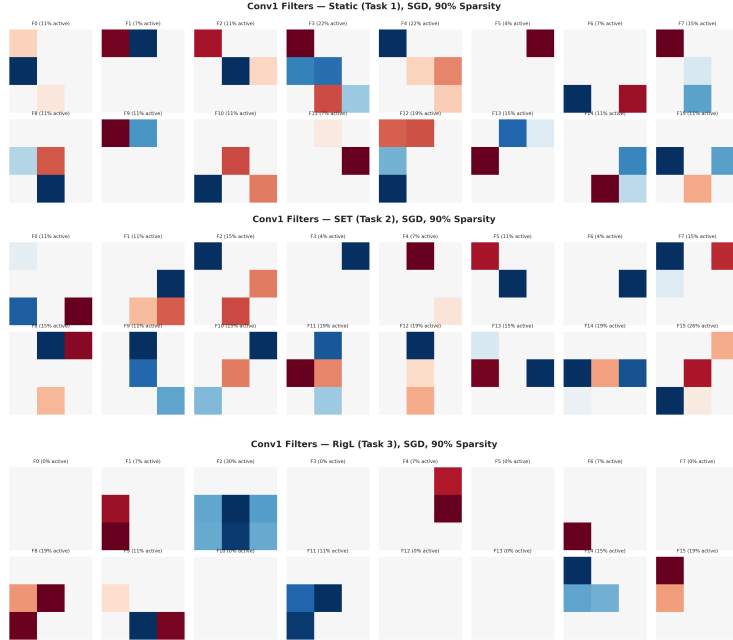


Figure 8: Conv1 filter visualizations for Static Sparse Training, SET, and RigL, all trained with SGD at 90% sparsity.

Figure 8 shows the conv1 filters for Static, SET, and RigL respectively. Across all three methods, the extreme sparsity of the stem layer means that most filters retain only one or two active weights out of nine, making it difficult to observe the classical edge-detector patterns typically learned by dense networks. Nevertheless, meaningful differences emerge across methods.

The static sparse filters show the most consistent distribution of active weights: nearly all 16 filters have between 7% and 22% of their weights active, and many exhibit both positive and negative active weights at different spatial positions — for example, F3 and F4 show opposing red and blue activations, suggesting the emergence of rudimentary contrast-detecting patterns. This diversity is consistent with a fixed random mask that distributes active connections broadly across all filters from initialization.

The SET filters show a similar range of per-filter activity percentages (4–26%), and the active weights are likewise distributed across both positive and negative values. However, F0 retains very weak near-zero activations despite being listed as 11% active, suggesting that some randomly re-grown connections failed to develop meaningful weight magnitudes. The highest-activity filter, F15

Table 2: Per-layer sparsity statistics for RigL (SGD, 90% target sparsity).

Layer	Stage	Parameters	Measured Sparsity
conv1	stem	432	90.28%
layer1.0.conv1	layer1	2,304	90.58%
layer1.0.conv2	layer1	2,304	89.24%
layer1.1.conv1	layer1	2,304	90.02%
layer1.1.conv2	layer1	2,304	90.15%
layer1.2.conv1	layer1	2,304	90.36%
layer1.2.conv2	layer1	2,304	90.06%
layer2.0.conv1	layer2	4,608	90.47%
layer2.0.conv2	layer2	9,216	90.18%
layer2.0.shortcut.0	layer2	512	91.60%
layer2.1.conv1	layer2	9,216	89.92%
layer2.1.conv2	layer2	9,216	89.90%
layer2.2.conv1	layer2	9,216	89.76%
layer2.2.conv2	layer2	9,216	90.04%
layer3.0.conv1	layer3	18,432	89.88%
layer3.0.conv2	layer3	36,864	90.01%
layer3.0.shortcut.0	layer3	2,048	89.55%
layer3.1.conv1	layer3	36,864	90.12%
layer3.1.conv2	layer3	36,864	89.95%
layer3.2.conv1	layer3	36,864	90.12%
layer3.2.conv2	layer3	36,864	90.13%
linear	fc	640	89.69%

at 26% active, shows a distributed pattern of positive and negative weights spanning multiple spatial locations, indicating that at least some filters were able to consolidate useful representations under dynamic rewiring.

The RigL filters present the most striking pattern. The active weight distribution is highly uneven: seven of the sixteen filters (F0, F3, F5, F7, F10, F12, F13) are completely dead with 0% active connections, while filter F2 retains 30% of its weights — the highest of any filter across all three methods — and shows a strong, coherent blue pattern concentrated in a left-right structure. This consolidation effect, where gradient-guided growth concentrates capacity into fewer but more active filters while leaving others entirely pruned, reflects RigL’s tendency to reinforce connections that are already contributing to loss reduction. While this can lead to more specialized individual filters, the number of dead filters in the stem layer suggests that at 90% sparsity, RigL’s aggressive gradient-guided selection causes some filters to be starved of connections entirely.

Table 2 reports per-layer sparsity for RigL at 90%. The measured sparsity values across all 22 layers range from 89.24% to 91.60%, remaining within approximately 1–2 percentage points of the 90% target throughout.

4.5 CONNECTIVITY EVOLUTION (RIGL)

To analyze how the sparse topology evolves during training under RigL, we tracked the per-stage topology change rate across all 15 epochs at SGD with 90% sparsity. The topology change rate at each epoch is defined as the fraction of connections that were flipped — either from active to inactive (pruned) or inactive to active (grown) — by the topology update step. This quantity directly measures how aggressively the network rewires itself at each update.

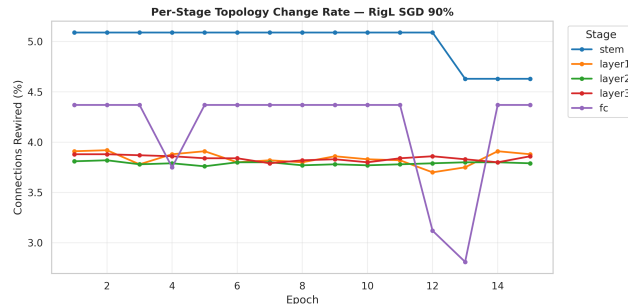


Figure 9: Per-stage change rate for RigL across epochs at SGD with 90% sparsity

Figure 9 shows the per-stage change rate for RigL across epochs. The rate stabilizes early and remains consistent at approximately 3.8–5.1% of connections rewired per epoch, depending on the stage. The stem layer (conv1) exhibits a slightly higher and more stable rate due to its smaller parameter count and higher sensitivity to fixed-count updates. Across all stages, there is no clear trend over time, indicating that RigL continues to identify useful connections throughout training rather than converging to a fixed topology early. This sustained rewiring reflects RigL’s ability to adaptively refine connectivity based on the loss landscape.

5 CONCLUSION

We studied sparse neural network training from scratch using Static Sparse Training, SET, and RigL on ResNet-20 with CIFAR-10 across 90%, 95%, and 98% sparsity. The results highlight clear trade-offs between fixed and dynamic connectivity, as well as strong interactions with optimizer choice.

First, Static Sparse Training provides a strong and stable baseline, often matching or outperforming dynamic methods when optimization remains well-behaved. Its fixed connectivity allows consistent feature development, avoiding the instability introduced by frequent topology changes.

Among dynamic methods, RigL consistently outperforms SET by a large margin (≈ 24 pp), demonstrating the importance of gradient-guided rewiring over random regrowth. While SET introduces noisy updates and unstable training dynamics—including complete failure at 95% sparsity for SGD—RigL is able to maintain structured and task-relevant connectivity. However, at extreme sparsity (98%), even RigL can become unstable depending on the optimizer.

Second, optimizer choice plays a critical role. Adam is more robust under dynamic sparsity due to its adaptive updates, maintaining stable performance even as connectivity evolves. In contrast, SGD performs well in static settings but becomes sensitive to frequent rewiring, leading to instability and generalization collapse at high sparsity.

Finally, performance generally degrades as sparsity increases, with a sharp drop beyond 95% indicating the removal of essential representational capacity rather than redundant parameters. While occasional gains at extreme sparsity suggest a weak regularization effect, these outcomes are inconsistent and highly dependent on training dynamics.

Overall, RigL with Adam provides the most reliable performance under dynamic sparsity, while Static Sparse Training remains a competitive and stable baseline. These results suggest that both gradient-informed connectivity updates and optimizer robustness are key to effective sparse training at high sparsity levels.

REFERENCES

- Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. Rigging the lottery: Making all tickets winners. *ICML*, 2020.
- Decebal Constantin Mocanu et al. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature Communications*, 2018.